DBMS LAB

Siddharth Karmokar

- 123CS0061

- ## Part A – First Normal Form (1NF)

This part explores the **atomicity rule of 1NF**, where multi-valued attributes (like a list of products in one column) are split into individual rows. Data redundancy is reduced, and querying becomes easier.

```
create table orders_raw(
    orderid number primary key,
    customer varchar2(50),
    products varchar2(50)
);

insert into orders_raw values (1, 'alice', 'pen, notebook');
insert into orders_raw values (2, 'bob', 'pencil');
insert into orders_raw values (3, 'alice', 'eraser, pen');
```

Script Output | Query Result

All Rows Fetched: 3 in 0.008 seconds

	ORDERID	CUSTOMER	PRODUCTS
1	1	alice	pen, notebook
2	2	bob	pencil
3	3	alice	eraser, pen

```
insert into orders_lnf values (1, 'alice', 'pen');
insert into orders_lnf values (1, 'alice', 'notebook');
insert into orders_lnf values (1, 'alice', 'eraser');
insert into orders_lnf values (1, 'alice', 'pen');
insert into orders_lnf values (2, 'bob', 'pencil');

select * from orders_lnf;
select * from orders_raw;
```

Script Output | Query Result

All Rows Fetched: 5 in 0.003 seconds

	ORDERID	CUSTOMER	PRODUCTS
1	1	alice	pen
2	1	alice	notebook
3	1	alice	eraser
4	1	alice	pen
5	2	bob	pencil

- ## Part B – Second Normal Form (2NF)

We examine a table that had a **partial dependency** where a non-key attribute (StudentName) depended only on part of the composite primary key (StudentID, CourseID). After decomposition into 2NF, we split student and enrollment info into separate tables.

```
insert into enroll values (1, 'dbms', 'alice', 85);
insert into enroll values (1, 'sql', 'alice', 90);
insert into enroll values (2, 'ai', 'bob', 88);
insert into enroll values (2, 'ml', 'bob', 92);

select * from enroll;
```

Script Output | Query Result | Query Result 1

All Rows Fetched: 4 in 0.011 seconds

	STUDENTID	COURSEID	STUDENTNAME	MARKS
1	1	dbms	alice	85
2	1	sql	alice	90
3	2	ai	bob	88
4	2	ml	bob	92

```
update enroll
set marks = 70
where studentid = 1;

select * from enroll;
```

Script Output | Query Result | Query Result 1 | Query Result 2

All Rows Fetched: 4 in 0.004 seconds

	STUDENTID	COURSEID	STUDENTNAME	MARKS
1	1	dbms	alice	70
2	1	sql	alice	70
3	2	ai	bob	88
4	2	ml	bob	92

```
    primary key (studentid, courseid)
);

insert into enroll_2nf values (1, 'dbms', 'alice', 85);
insert into enroll_2nf values (1, 'sql', 'alice', 90);
insert into enroll_2nf values (2, 'ai', 'bob', 88);
insert into enroll_2nf values (2, 'ml', 'bob', 92);

select * from enroll_2nf;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3

SQL | All Rows Fetched: 4 in 0.004 seconds

	STUDENTID	COURSEID	STUDENTNAME	MARKS
1	1	dbms	alice	85
2	1	sql	alice	90
3	2	ai	bob	88
4	2	ml	bob	92

```
update enroll_2nf
set marks = 71
where studentid = 1 and courseid = 'sql';

select * from enroll_2nf;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5

SQL | All Rows Fetched: 4 in 0.004 seconds

	STUDENTID	COURSEID	STUDENTNAME	MARKS
1	1	dbms	alice	85
2	1	sql	alice	71
3	2	ai	bob	88
4	2	ml	bob	92

- Part C – Third Normal Form (3NF)

This scenario demonstrates a **transitive dependency** (`EmpID → DeptID → DeptName`). To achieve 3NF, we created separate tables for employees and departments, ensuring that each non-key attribute only depends on the primary key.

```
);

insert into department values (101, 'anita', 'd1', 'hr');
insert into department values (102, 'meena', 'd2', 'it');
insert into department values (103, 'rakesh', 'd1', 'hr');

select * from department;
drop table department;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6 | Query Result 7

SQL | All Rows Fetched: 3 in 0.003 seconds

	EMPID	EMPNAME	DEPTID	DEPTNAME
1	101	anita	d1	hr
2	102	meena	d2	it
3	103	rakesh	d1	hr

```
update department
set deptname = 'human resources'
where empid = 101;

select * from department;
drop table department;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6 | Query Result 7 | Query Result 8

SQL | All Rows Fetched: 3 in 0.001 seconds

	EMPID	EMPNAME	DEPTID	DEPTNAME
1	101	anita	d1	human resources
2	102	meena	d2	it
3	103	rakesh	d1	hr

```
create table department(
    deptid varchar2(50) primary key,
    deptname varchar2(50)
);

insert into department values ('d1', 'hr');
insert into department values ('d2', 'it');

select * from department;
```

Script Output | Query Result

SQL | All Rows Fetched: 2 in 0.003 seconds

	DEPTID	DEPTNAME
1	d1	hr
2	d2	it

```sql
create table employee_3nf(
    empid number primary key,
    empname varchar2(50),
    deptid varchar2(50),
    foreign key(deptid) references department(deptid)
);

insert into employee_3nf values (101, 'anita', 'd1');
insert into employee_3nf values (102, 'meena', 'd2');
insert into employee_3nf values (103, 'rakesh', 'd1');

select * from employee_3nf;

select * from department;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3

SQL | All Rows Fetched: 3 in 0.003 seconds

	EMPID	EMPNAME	DEPTID
1	101	anita	d1
2	102	meena	d2
3	103	rakesh	d1

```sql
update department
set deptname = 'human resources'
where deptid = 'd1';

select * from department;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4

SQL | All Rows Fetched: 2 in 0.006 seconds

	DEPTID	DEPTNAME
1	d1	human resources
2	d2	it

```sql
select * from employee_3nf;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5

SQL | All Rows Fetched: 3 in 0.001 seconds

	EMPID	EMPNAME	DEPTID
1	101	anita	d1
2	102	meena	d2
3	103	rakesh	d1

- # Part D – Boyce–Codd Normal Form (BCNF)

This part addresses a **mutual dependency** between `Professor` and `Room` (i.e., `Professor → Room` and `Room → Professor`), which violates BCNF. To fix it, we decomposed the original table into:

- A `PROFESSOR_ROOM` table (mapping professors to rooms),
- A `TEACHING_BCNF` table (storing subject assignments).

This ensures that **every determinant is a candidate key**, eliminating redundancy and anomalies.

```sql
create table teaching(
    professor varchar2(100),
    subject varchar2(50),
    room varchar2(50)
);

insert into teaching values ('Dr.Rao', 'dbms', 'room101');
insert into teaching values ('Dr.Rao', 'sql', 'room101');
insert into teaching values ('Dr.Meena', 'ai', 'room202');
insert into teaching values ('Dr.Meena', 'ml', 'room202');

select * from teaching;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5

SQL | All Rows Fetched: 4 in 0.003 seconds

	PROFESSOR	SUBJECT	ROOM
1	Dr.Rao	dbms	room101
2	Dr.Rao	sql	room101
3	Dr.Meena	ai	room202
4	Dr.Meena	ml	room202

```sql
update teaching
set room = 'room105'
where professor = 'Dr.Rao' and subject = 'sql';

select * from teaching;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6

SQL | All Rows Fetched: 4 in 0.002 seconds

	PROFESSOR	SUBJECT	ROOM
1	Dr.Rao	dbms	room101
2	Dr.Rao	sql	room105
3	Dr.Meena	ai	room202
4	Dr.Meena	ml	room202

```sql
create table professor_room(
    professor varchar2(50) primary key,
    room varchar2(50)
);

insert into professor_room values ('Dr.Rao', 'room101');
insert into professor_room values ('Dr.Meena', 'room202');

select * from professor_room;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6 | Query Result 7

All Rows Fetched: 2 in 0.01 seconds

	PROFESSOR	ROOM
1	Dr.Rao	room101
2	Dr.Meena	room202

```sql
create table teaching_bcnf(
    professor varchar2(50),
    subject varchar2(50),
    primary key(professor, subject),
    foreign key(professor) references professor_room(professor)
);

insert into teaching_bcnf values ('Dr.Rao', 'dbms');
insert into teaching_bcnf values ('Dr.Rao', 'sql');
insert into teaching_bcnf values ('Dr.Meena', 'ai');
insert into teaching_bcnf values ('Dr.Meena', 'ml');

select * from teaching_bcnf;
select * from professor_room;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6 | Query Result 7 | Query Result 8

All Rows Fetched: 4 in 0.006 seconds

	PROFESSOR	SUBJECT
1	Dr.Rao	dbms
2	Dr.Rao	sql
3	Dr.Meena	ai
4	Dr.Meena	ml

```sql
update professor_room
set room = 'room105'
where professor = 'Dr.Rao';

select * from professor_room;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6 | Query Result 7 | Query Result 8 | Query Result 9

All Rows Fetched: 2 in 0.008 seconds

	PROFESSOR	ROOM
1	Dr.Rao	room105
2	Dr.Meena	room202

```sql
select * from teaching_bcnf;
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4 | Query Result 5 | Query Result 6 | Query Result 7 | Query Result 8 | Query Result 9 | Query Result 10

All Rows Fetched: 4 in 0.001 seconds

	PROFESSOR	SUBJECT
1	Dr.Rao	dbms
2	Dr.Rao	sql
3	Dr.Meena	ai
4	Dr.Meena	ml